

Primary Index

1 What is a Primary Index?

A **Primary Index** is an indexing mechanism specified on an ordered data file where the search key defines the primary key/unique ordering of the file.

★ **GOLDEN RULE**
Primary Index = Ordered + Key (Unique)

Condition	Meaning in Database Context
Ordered Data	Data records are physically sorted in ascending or descending order on disk.
Key Attribute	The search key attribute contains strictly unique values (no duplicate keys allowed).

2 Example Illustration

Consider a Student Table stored with the following schema and data:

Roll No. (Key)	Name	Age	Address
1	A	19	Delhi
2	B	20	Pune
3	C	18	Mumbai
4	D	19	Hubli
5	E	20	Koppal

Here, **Roll No.** is strictly **Unique** ✓ and physically **Ordered** ✓. Therefore, it serves as an ideal search key for a Primary Index.

3 Primary Index Structure

Each index entry in a Primary Index table consists of exactly two fields:

- 1. **Key Value:** First key value of the corresponding data block.
- 2. **Block Pointer:** Points directly to the target disk data block.

[Key Value | Pointer]

4 Real-Life Analogy

Train System Search (IRCTC) Train Name: <i>Rajdhani Express</i> – Multiple trains exist with this name (Non-Unique / Non-Key). Train Number: 12951 – Unique identification number for	Student Database Search Student Name: Duplicate names can exist across records (Non-Key). Roll Number: Unique identifier assigned to each student
--	--

6 Anchor Records

An **Anchor Record** (or block anchor) is the first record in a data block that serves as the representative key for that entire block in the sparse index structure.

Formula: Total Number of Index Entries = Total Number of Data Blocks (B)

7 Why is Primary Index Sparse?

If a database contains 10,000,000 records, creating a dense entry for every record results in an unnecessarily massive index table. Using a sparse primary index provides major architectural benefits:

- **Fewer Index Entries:** Equal to block count (B) instead of record count (N).
- **Reduced Disk Space:** Index table easily fits in main memory (RAM).
- **Faster Binary Search:** Fewer index blocks to traverse during search.
- **Lower I/O Overhead:** Minimizes expensive disk block transfers.

8 Searching Using Primary Index

To locate a target record with search key value K :

1. Perform a binary search on the ordered index entries to find consecutive index keys K_i and K_{i+1} such that $K_i \leq K < K_{i+1}$.
2. Follow the block pointer associated with K_i to retrieve the data block.
3. Scan the retrieved data block to find the required record.

Execution Trace Examples:

- **Search Record 3:** Since $1 \leq 3 < 5$, pointer directs to **Block 1**. Search inside Block 1.
- **Search Record 8:** Since $5 \leq 8 < 9$, pointer directs to **Block 2**. Search inside Block 2.

9 Time Complexity Analysis

Because the primary index table is kept sorted, we apply **Binary Search** over the index entries.

Let N be the total number of blocks in the index table. Time complexity is given by:

Total Block Accesses = $\log_2 N + 1$

Overall Search Complexity: **$O(\log_2 N)$**

Why +1?

- $\log_2 N$ accesses to search the index
- + 1 final access to fetch the data block

10 Primary Index vs. Dense Index

Feature	Primary Index	Dense Index
Data Ordering	Must be Sorted / Ordered	Can be Unordered or Ordered
Search Key	Unique Key Attribute	Key or Non-Key Attribute
Sparsity	Sparse (1 entry per Data Block)	Dense (1 entry per Record)
Total Entries	Equal to Data Block Count (B)	Equal to Total Record Count (R)
Search Effort	$\log_2 B + 1$	$\log_2 R + 1$